

Apresentação da Disciplina e Fundamentos de C

COMP0512 — Programação C · Turma T02 · 2026.2

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

A Disciplina

Revisão de Fundamentos da Linguagem C

Como a disciplina funciona: conteúdo, metodologia e avaliação



Datas importantes do semestre 2026.2



Revisão prática dos fundamentos de C: prever, executar e modificar código

A Disciplina

Disciplina	Programação C (COMP0512)
Turma / Horário	T02 / 24T34 (segundas e quartas à tarde)
Carga horária	60h (30h teórica + 30h prática)
Pré-requisito	COMP0496
Período letivo	17/08/2026 a 21/12/2026
Professor	André Yoshiaki Kashiwabara
Contato	andreyoshiaki@dcomp.ufs.br

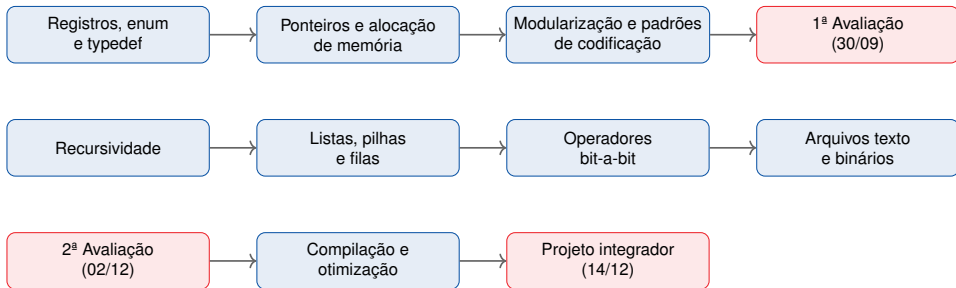
Objetivo

Capacitar o aluno a projetar e implementar programas em linguagem C que empreguem **estruturas de dados fundamentais, alocação dinâmica de memória, modularização e manipulação de arquivos**, seguindo boas práticas e padrões de codificação.

Em outras palavras

Ao final do semestre, você será capaz de construir programas C completos, organizados em múltiplos arquivos, que gerenciam memória corretamente e persistem dados em disco.

O que veremos no semestre



Data	O quê	Peso
30/09/2026	1ª Avaliação (P1) — prova escrita individual	2
02/12/2026	2ª Avaliação (P2) — prova escrita individual	2
14/12/2026	Apresentação do projeto integrador (PJ)	1
16/12/2026	Avaliação de reposição	—
21/12/2026	Encerramento e consolidação das notas	—

Sem aula

07/09 (Independência), 12/10 (N. Sra. Aparecida), 28/10 (Dia do Servidor Público) e 02/11 (Finados).

Média final

$$MF = \frac{2 \cdot P1 + 2 \cdot P2 + PJ}{5}$$

- Aprovação: $MF \geq 5,0$ e frequência mínima de 75% da carga horária.
- Projeto integrador: aplicação em C integrando os conteúdos da disciplina.
- Reposição (16/12): para quem faltou a uma das avaliações por motivo justificado; a nota substitui a da avaliação perdida.
- Exercícios e estudos dirigidos complementam a preparação ao longo do semestre.

Como serão as aulas

- Presenciais, preferencialmente em laboratório.
- Teoria com slides + implementação de código ao vivo.
- Exercícios em sala e estudos dirigidos para casa.
- Use a IDE de sua preferência.

Canais

- **SIGAA**: envio de atividades e trabalhos.
- E-mail:
`andreyoshiaki@dcomp.ufs.br`

- KERNIGHAN, B. W.; RITCHIE, D. M. *C: a linguagem de programação — padrão ANSI*. Campus, 1989.
- SCHILDT, H. *C completo e total*. 3. ed. Makron Books, 1997.
- TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J. *Estruturas de dados usando C*. Pearson Makron Books, 1995.

Complementar (destaques)

BACKES, *Linguagem C: completa e descomplicada*; FEOFILOFF, *Algoritmos em linguagem C*; DEITEL; DEITEL, *C: como programar*.

Revisão de Fundamentos da Linguagem C

Problema:

Precisamos de uma linguagem que permita controlar a memória e o hardware de perto — sistemas operacionais, sistemas embarcados, bancos de dados e interpretadores exigem esse controle.

Solução:

C oferece acesso direto à memória (ponteiros), tipos simples e previsíveis e compilação para código de máquina eficiente — é a base de Linux, Python, Git e de boa parte da infraestrutura computacional moderna.

Nesta disciplina

C é a ferramenta para entender **como os dados vivem na memória** — conhecimento que vale para qualquer linguagem.



O contrato de hoje

Vocês encontram o código **antes** da teoria — a teoria chega como resposta às perguntas que o código levantar.

Um programa para começar: a média da turma



```
1  #include <stdio.h>
2  #define TAM 5
3
4  double media(double v[], int n) {
5      double soma = 0.0;
6      for (int i = 0; i < n; i++)
7          soma += v[i];
8      return soma / n;
9  }
```

```
10  int main(void) {
11      double notas[TAM];
12      printf("Digite %d notas: ", TAM);
13      for (int i = 0; i < TAM; i++)
14          scanf("%lf", &notas[i]);
15      double m = media(notas, TAM);
16      printf("Media: %.2f\n", m);
17      if (m >= 5.0)
18          printf("Turma aprovada\n");
19      else if (m >= 3.0)
20          printf("Turma em reposicao\n");
21      else
22          printf("Turma reprovada\n");
23      return 0;
24  }
```

Em duplas, anotem a previsão (2–3 min) — sem computador!

1. Com as notas 7.0 8.0 6.5 9.0 5.5, o que aparece na tela, linha por linha?
2. E com as notas 2.0 3.0 1.0 4.0 2.5?
3. A média é impressa com quantas casas decimais? Por quê?

Regra do jogo

Não existe previsão “feia”: previsão errada é o ponto de partida da investigação. Anote exatamente o que você espera ver na tela.

```
$ gcc media_turma.c -o media_turma
$ ./media_turma
Digite 5 notas: 7.0 8.0 6.5 9.0 5.5
Media: 7.20
Turma aprovada
```

- Sua previsão bateu com a saída real? Linha por linha?
- Se divergiu, anote **onde** divergiu: é aí que mora a revisão de hoje.
- Teste também o caso (b) e confira a classificação impressa.

O que o gcc acabou de fazer?



Na prática

`gcc media_turma.c -o media_turma` executou as quatro etapas de uma vez;
`./media_turma` rodou o programa.

O que `#include`, `main` e `return 0` fazem?



```
1  #include <stdio.h>    /* traz declaracoes de printf e scanf */
2
3  int main(void) {      /* funcao principal: inicio da execucao */
4      /* ... corpo do programa ... */
5      return 0;         /* 0 indica sucesso ao sistema */
6  }
```

- `#include` traz declarações de bibliotecas — sem ele, `printf` e `scanf` seriam nomes desconhecidos.
- Todo programa começa em `main`, esteja ela onde estiver no arquivo.
- Cada comando termina em `;` e blocos ficam entre `{ }`.

Tipo	Conteúdo	Tamanho típico	Exemplo
<code>int</code>	inteiros	4 bytes	42
<code>float</code>	reais (precisão simples)	4 bytes	3.14f
<code>double</code>	reais (precisão dupla)	8 bytes	2.718
<code>char</code>	um caractere	1 byte	'A'
<code>long</code>	inteiros grandes	8 bytes	1000000L

Atenção

Os tamanhos dependem da plataforma — use `sizeof` para verificar.

Guarde esta pergunta

E se soma fosse `int`? Testaremos daqui a pouco, modificando o programa.

Por que ¬as[i] no scanf?



```
1 printf("Digite %d notas: ", TAM);  
2 scanf("%lf", &notas[i]);          /* & = endereço da variável */  
3 printf("Media: %.2f\n", m);
```

- printf recebe o **valor**; scanf precisa do **endereço** onde guardar o que for lido — por isso o &.
- Formatos: %d (int), %f/%lf (real), %c (char), %s (string).
- %.2f imprime com 2 casas decimais — a resposta da pergunta (c).
- O & já é um ponteiro disfarçado — tema das próximas semanas.

Como o programa decide “aprovada”?

```
1  if (m >= 5.0)      printf("Turma aprovada\n");
2  else if (m >= 3.0) printf("Turma em reposicao\n");
3  else               printf("Turma reprovada\n");
4
5  /* alternativa para menus e opcoes discretas */
6  switch (opcao) {
7      case 1: printf("Inserir\n"); break;
8      case 2: printf("Remover\n"); break;
9      default: printf("Opcao invalida\n");
10 }
```

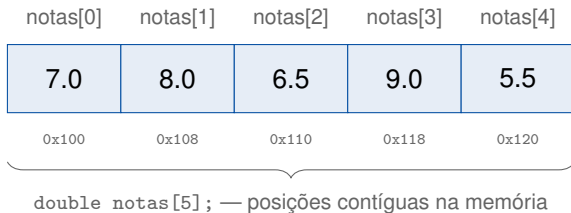
- As condições são testadas **em ordem**; a primeira verdadeira vence.
- Com $m = 7.20$, o teste $m \geq 5.0$ já decide: “Turma aprovada”.

Quantas vezes cada laço executa?



```
1  /* for: numero de repeticoes conhecido */
2  for (int i = 0; i < TAM; i++)
3      scanf("%lf", &notas[i]);      /* executa TAM vezes */
4
5  /* while: repete enquanto a condicao vale */
6  int n = 32;
7  while (n > 1)
8      n = n / 2;
9
10 /* do-while: executa ao menos uma vez (menus!) */
11 int opcao;
12 do {
13     scanf("%d", &opcao);
14 } while (opcao != 0);
```

O que acontece na memória com notas?



- Índices começam em **0**; o último é `TAM - 1`.
- Cada `double` ocupa 8 bytes — os endereços avançam de 8 em 8.
- **C não verifica limites**: acessar `notas[5]` é erro silencioso.
- Strings são vetores de `char` terminados em `'\0'`.

Por que `media` recebe o vetor e o tamanho?

```
1  double media(double v[], int n) {    /* definida antes do main */
2      double soma = 0.0;
3      for (int i = 0; i < n; i++)
4          soma += v[i];
5      return soma / n;
6  }
```

- Parâmetros comuns são **copiados** (passagem por valor).
- Vetores são a exceção: a função recebe o **endereço** do primeiro elemento — por isso o tamanho `n` precisa ir junto.
- Para definir a função depois do `main`, declare antes um **protótipo**: `double media(double v[], int n);`

Tarefas, em ordem crescente de desafio

1. Mude TAM para 8 e ajuste o programa para continuar correto. O que precisou mudar? O que mudou sozinho?
2. Troque o tipo de `soma` para `int`, execute com as mesmas notas de antes e explique a nova saída.
3. Escreva `double maior(double v[], int n)` e imprima também a maior nota da turma.

Dica

Depois de cada alteração: recompila, execute e **preveja antes de olhar** a nova saída.

Desafio

Escreva um programa que lê as notas P1, P2 e PJ de um aluno, calcula

$$MF = \frac{2 \cdot P1 + 2 \cdot P2 + PJ}{5}$$

e imprime **Aprovado** ($MF \geq 5,0$), **Reposição** ($3,0 \leq MF < 5,0$) ou **Reprovado** — exatamente a regra de avaliação desta disciplina.

Terminou? Estenda

Leia as notas de vários alunos, guarde as médias finais num vetor e conte quantos foram aprovados.

O que revisamos hoje

- Compilação: do `.c` ao executável com `gcc`.
- Tipos básicos, variáveis e operadores.
- Entrada e saída com `printf` e `scanf`.
- Decisão (`if`, `switch`) e repetição (`for`, `while`, `do-while`).
- Funções com passagem por valor.
- Vetores e strings: posições contíguas, índice a partir de 0.

Como revisamos

Prevendo, executando, investigando, modificando e criando.

- **Próxima aula (19/08):** tipos de dados, estruturas de dados e tipos abstratos de dados (conceitos).
- Fazer o tutorial guiado (Jupyter Notebook) do Classroom — ele continua as tarefas de modificação e o desafio para quem não terminou em sala.
- Garantir um ambiente C funcionando: `gcc` ou IDE de preferência.
- Revisar os capítulos iniciais de Kernighan & Ritchie ou Backes.
- Dúvidas: `andreyoshiaki@dcomp.ufs.br` ou no início da próxima aula.



André Backes.

Linguagem C: completa e descomplicada.

Elsevier, Rio de Janeiro, 2013.



Randal E. Bryant and David R. O'Hallaron.

Computer Systems: A Programmer's Perspective.

Pearson, Boston, 3 edition, 2015.

Seção 3.9: alinhamento de dados e layout de structs.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Algoritmos: teoria e prática.

Elsevier, Rio de Janeiro, 3 edition, 2012.



Paul Deitel and Harvey Deitel.

C: como programar.

Pearson Prentice Hall, São Paulo, 6 edition, 2011.



Ulrich Drepper.

What Every Programmer Should Know About Memory.

Red Hat, Inc., 2007.

Seção 6.2: efeito da ordem de percurso de matrizes sobre a cache.



Ulrich Drepper.

How to write shared libraries.

Documento técnico, 2011.

Versão 4.1.2.



Paulo Feofiloff.

Algoritmos em linguagem C.

Elsevier, Rio de Janeiro, 2009.



Free Software Foundation.

Gnu coding standards.

Free Software Foundation.

Acesso em 2026.



Free Software Foundation.

Using the GNU compiler collection — options to request or suppress warnings.

Manual do GCC.

Acesso em 2026.



ISO/IEC.

Iso/iec 9899:2018 — information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seção 6.7.2.1: structure and union specifiers.



Brian W. Kernighan and Rob Pike.

The Practice of Programming.
Addison-Wesley, Reading, MA, 1999.
Cap. 1, “Style”: nomes, expressões, consistência, números mágicos e comentários.



Brian W. Kernighan and Dennis M. Ritchie.

C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.



K. N. King.

C Programming: A Modern Approach.

W. W. Norton, New York, 2 edition, 2008.

Capítulo 16: estruturas, uniões e enumerações.



Steve McConnell.

Code Complete: A Practical Handbook of Software Construction.

Microsoft Press, Redmond, WA, 2 edition, 2004.

Cap. 11 (nomes de variáveis), cap. 31 (layout e estilo) e cap. 32 (código autodocumentado).



Robert Mecklenburg.

Managing Projects with GNU Make.

O'Reilly Media, 3 edition, 2004.



Herbert Schildt.

C completo e total.

Makron Books, São Paulo, 3 edition, 1997.



Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.

AddressSanitizer: A fast address sanity checker.

In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.

Ferramenta usada no tutorial para detectar vazamentos e uso após free.



Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.

Estruturas de dados usando C.

Pearson Makron Books, São Paulo, 1995.



The Linux Kernel Developers.

Linux kernel coding style.

Documentação do kernel Linux, [Documentation/process/coding-style.rst](#).

Acesso em 2026.



The LLVM Project.

Clang-format style options.

Documentação do Clang.

Acesso em 2026.



Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles.

Dynamic storage allocation: A survey and critical review.

In *International Workshop on Memory Management (IWMM)*, pages 1–116. Springer, 1995.

Panorama de estratégias de alocação dinâmica e fragmentação.



Nivio Ziviani.

Projeto de algoritmos: com implementações em Pascal e C.

Cengage Learning, São Paulo, 3 edition, 2011.